Report for AI Capstone Homework 3: A Robot Navigation Framework

FU, YONG-WEI (112550107)

April 27, 2026

1 Implementation

1.1 Code explanation

My pipeline is composed of several modules, and there are four of them I want to dig into:

1.1.1 load_and_filter_map()

```
def load_and_filter_map(point_path: str, color_path: str,
height_limit: float = 1.5):
    points = np.load(point_path)
    colors = np.load(color_path)

    min_xyz = np.min(points, axis=0)
    points = points - min_xyz
    points = points * SCALE_FACTOR

    colors_int = np.round(colors).astype(int)
    y_coords = points[:, 1]
    is_floor = np.all(colors_int == FLOOR_COLOR, axis=1)
    is_within_height = (y_coords < height_limit) & (y_coords > 0.2)

    walkable_mask = is_floor
    obstacle_mask = (~is_floor) & is_within_height

    ...
```

Figure 1: Implementation of the 3D point cloud processing and filtering logic.

The `load_and_filter_map()` function converts raw 3D point cloud data into a 2D navigation grid.

- **Coordinate Transformation:** Raw points are shifted into the positive quadrant and then scaled using a `SCALE_FACTOR` to convert them into real-world units first.
- **Mask Filtering:** I filter points into two masks (`walkable_mask` and `obstacle_mask`) based on their semantic colors and vertical Y-axis coordinates. Specifically, I extracted the `walkable_mask` based strictly on the semantic floor color, and defined the `obstacle_mask` by identifying points that are **not** the floor and fall within a specific vertical range ($0.2m < y < 1.5m$). This ensures the robot ignores the ground it stands on while capturing all physical barriers that exist within its height clearance.
- **2D Projection:** The 3D (x, y, z) points are projected onto a 2D (x, z) grid with a resolution of $0.05m$ per pixel.

- **Morphological Processing:** I use `cv2.morphologyEx` with a kernel to fill gaps in the floor map (caused by sparse point cloud data). I also use `cv2.dilate` on the occupancy map to add a safety margin around obstacles, ensuring the RRT does not plan paths too close to walls.

1.1.2 pick_goal()

```python
def pick_goal(map_img, occupancy_map, floor_map) -> Tuple[str, Tuple[int, int]]:
    prompt = (

"Enter semantic destination ('rack', 'cooktop', 'sofa', 'cushion', 'stair'): "
    goal_prompt = input(prompt).strip().lower()

    if goal_prompt not in SEMANTIC_DICTS["colors"]:
        print(f"Goal '{goal_prompt}' is not valid.")
        sys.exit(1)

    # 1. Get the raw semantic pixels
    object_pixels = get_goal_pixels(map_img, SEMANTIC_DICTS["colors"], goal_prompt)

    # 2. Use the floor-aware logic to find a valid navigation point
    goal = find_navigable_goal(object_pixels, occupancy_map, floor_map)

    ...
```

Figure 2: Code snippet showing the goal selection and distance transform logic.

The `pick_goal()` function and its helpers translate a user's semantic text command into a precise, reachable coordinate on the 2D grid.

- **Semantic Color Matching:** The `get_goal_pixels()` function iterates through the normalized RGB values defined in `SEMANTIC_DICTS`. It identifies all pixels in the `map_img` that match the target furniture and returns it.
- **Distance Transform for Clearance:** To ensure the robot doesn't get stuck in a "dead zone" (pixels too close to obstacles to be reachable by RRT and the robot), I used `cv2.distanceTransform` to calculate the Euclidean distance from every free pixel to the nearest obstacle. By requiring a `SAFE_DISTANCE` of at least 5 pixels, the algorithm filters out narrow gaps and "edge pixels" that would otherwise cause navigation failures.
- **Centroid-based Navigation Point:** Instead of picking a random pixel on the object's surface, the algorithm calculates the **centroid** $(\bar{x}, \bar{z})$ of all identified pixels of that object. This provides a stable reference point in the "center" of the furniture.
- **Nearest Safe Floor Selection:** The final goal is determined by finding the pixel that is:
  1. Labeled as floor in the `floor_map`.
  2. Marked as free space in the `occupancy_map`.
  3. Verified to have sufficient clearance via the distance transform.

  The algorithm uses `np.hypot` to calculate the distance from the object's centroid to all "safe" floor candidates and selects the one with the minimum distance. This ensures the robot stops directly in front of the target furniture when navigating.

1.1.3 plan_path()

```python
def plan_path(start_coords: Tuple[int, int], goal_coords: Tuple[int, int], occupancy_map
: np.ndarray, step_size: int = 10, max_iter: int = 20000) -> List[Tuple[int, int]]:
    """
    Executes the RRT algorithm to find a navigable path.
    """
    start_node = Node(start_coords[0], start_coords[1])
    goal_node = Node(goal_coords[0], goal_coords[1])

    tree = [start_node]
    height, width = occupancy_map.shape

    for _ in range(max_iter):

# 1. Exploration vs Exploitation: Sample a random point, with a 10% probability to sample
the exact goal
        if random.random() < 0.2:
            rnd_node = Node(goal_node.x, goal_node.y)
        else:
            rnd_node = Node(random.randint(0, width - 1), random.randint(0, height - 1))

        # 2. Find the nearest existing node in the tree to this random point
        nearest_node = min(tree, key=lambda n: get_distance_between_nodes(n, rnd_node))
```

Figure 3: RRT path planning implementation featuring goal-biasing and collision checking.

The `plan_path()` function implements the core Rapidly-exploring Random Tree (RRT) algorithm to navigate the 2D occupancy grid. It iteratively grows a tree from the start position until it successfully connects to the target goal.

- **Biased Sampling (Exploration vs. Exploitation):** To balance the search between exploring the entire map and moving directly toward the target, the algorithm uses a 20% goal bias. In each iteration, there is a 20% chance the goal itself is sampled as the target point, while the remaining 80% of the time a random coordinate is chosen from the entire map area.
- **Nearest Neighbor Search:** For every sampled point, regardless of whether it's the goal or a random point, the algorithm identifies the existing node in the `tree` that is geographically closest to it. This is achieved by calculating the Euclidean distance between the sample and every node currently in the tree using the `get_distance_between_nodes` helper.
- **Steering Logic:** Once the nearest node is found, the algorithm "steers" from that node toward the sampled point. Instead of jumping directly to the sample, it creates a new node exactly one `step_size` (e.g., 10 pixels) away. This ensures incremental, controlled growth of the tree and prevents the path from jumping over obstacles.
- **Pixel-wise Collision Detection:** The `check_collision()` function performs a rigorous safety check. It iterates through every pixel along the straight line between the current tree node and the proposed new node. If any pixel along this line is marked as an obstacle in the `occupancy_map`, or if the path exceeds the map boundaries, the new node is discarded.
- **Goal Connection and Backtracking:** When a new node is successfully added to the tree, the algorithm checks if the goal is within one `step_size` again. If a collision-free path exists to the goal, the `goal_node` is appended to the tree. The final navigable route is then reconstructed by backtracking from the goal to the start using the `parent` pointers stored in each node. The resulting list is reversed to provide a sequential path from Start to Goal.

1.1.4 run_in_sim()

```python
def run_in_sim(start_world, world_path, goal_prompt):
    """
    Initializes the simulator and executes the calculated path.
    """
    start_x, start_z = start_world

    # Initialize the Habitat simulator at the translated world start position
    sim, agent, _ = init_sim(start_x=start_x, start_z=start_z)

    # Get the semantic index from the hardcoded dictionary for highlighting
    goal_idx = SEMANTIC_DICTS["indices"][goal_prompt]

    print(f"Executing movement toward {goal_prompt} (Index: {goal_idx})...")
    execute_waypoint_path(world_path, sim, agent, goal_idx)
```

Figure 4: Simulation execution logic and navigation control parameters.

The `run_in_sim()` function transforms the abstract list of waypoints into a sequence of discrete physical actions within the Habitat environment and displays the navigation result.

- **Simulator Initialization and Configuration:** The `init_sim()` function configures the Habitat simulator with three primary sensors: Color (RGB), Depth, and Semantic. The action space is defined with discrete commands: `MOVE_FORWARD`, `TURN_LEFT`, and `TURN_RIGHT`. The agent's starting state is set with the starting point that the user has specified.
- **Two-Phase Navigation Logic:** The `execute_waypoint_path()` function implements a "turn-then-move" strategy for each waypoint pair:
  1. **Rotation:** The algorithm calculates the `target_angle` to the next waypoint using `math.atan2(dx, dz)`. The agent then executes a series of 1-degree turns to align its heading with the target vector.
  2. **Translation:** Once aligned, the agent calculates the number of forward steps required to bridge the distance.
- **Precision Tuning with `MOVE_AMOUNT`:** A critical improvement I made in this implementation was reducing the `MOVE_AMOUNT` parameter from 0.1 to 0.02. In discrete simulators, a large step size can cause the agent to "overshoot" a waypoint, leading to cumulative errors in position. By utilizing a smaller step size, the agent achieves much higher granular control, ensuring it stays strictly on the planned RRT path even when navigating tight corners or narrow doorways.
- **Visual Feedback and Semantic Highlighting:** The `navigate_and_see()` function provides real-time visualization of the agent's progress. To verify the navigation goal, the function utilizes the **Semantic Sensor**. When the target object's `goal_index` is visible in the agent's field of view, the matching pixels are highlighted via a mask and overlaid with a red color using **Alpha Blending** (`cv2.addWeighted`). This creates a semi-transparent "glow" effect, providing visual confirmation that the robot has correctly reached the intended destination.

1.2 Result and Discussion

In this section, I present the results of the RRT path planning and the subsequent robot navigation for three distinct targets: **Sofa**, **Cooktop**, and **Rack**.

1.2.1 Case 1: Sofa

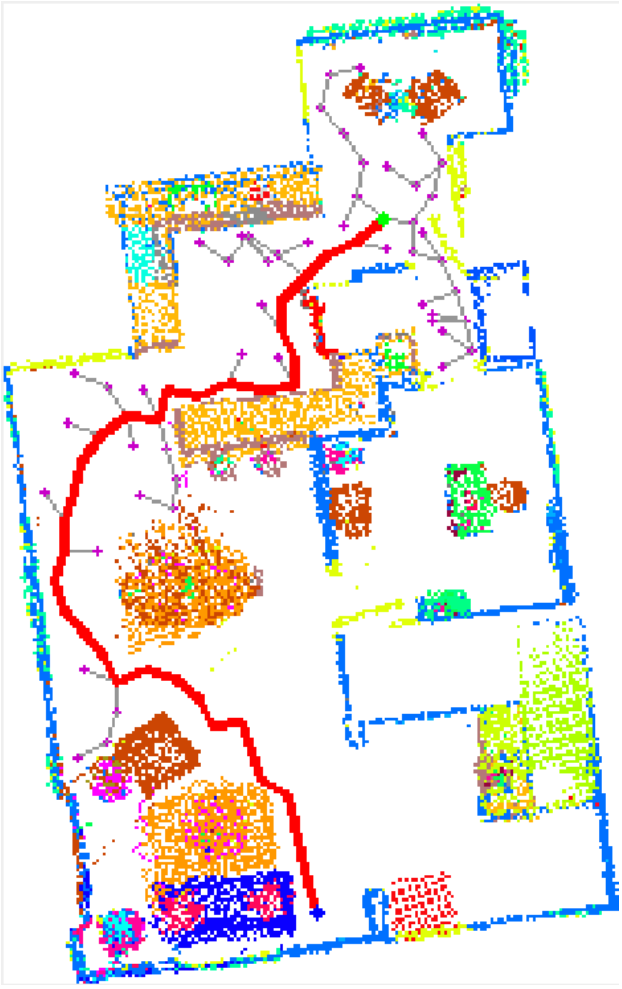

Figure 5: RRT path for the Sofa.

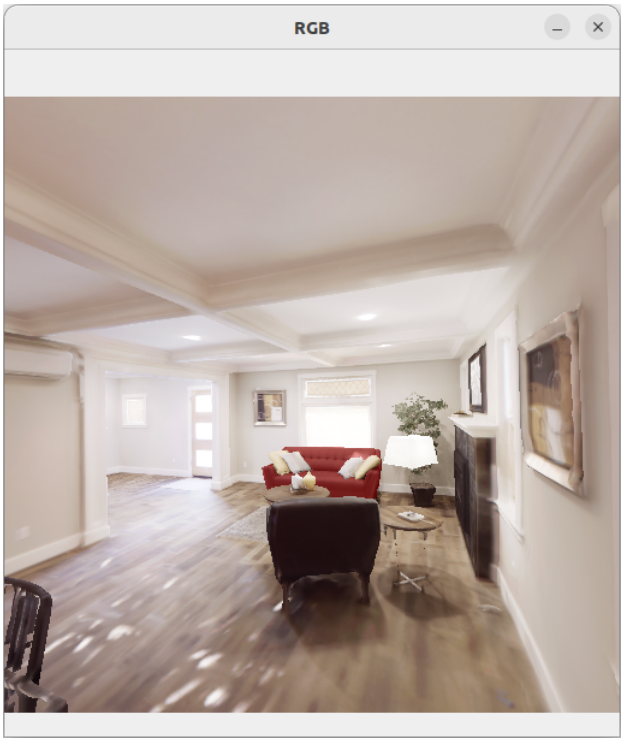

Figure 6: Simulation view with sofa highlighted.

Discussion: The RRT map reveals a successful exploration of the living area. The 20% goal bias efficiently directed the tree towards the sofa. Because the path to the sofa is relatively wide, the algorithm converged quickly.

1.2.2 Case 2: Navigation to the Cooktop

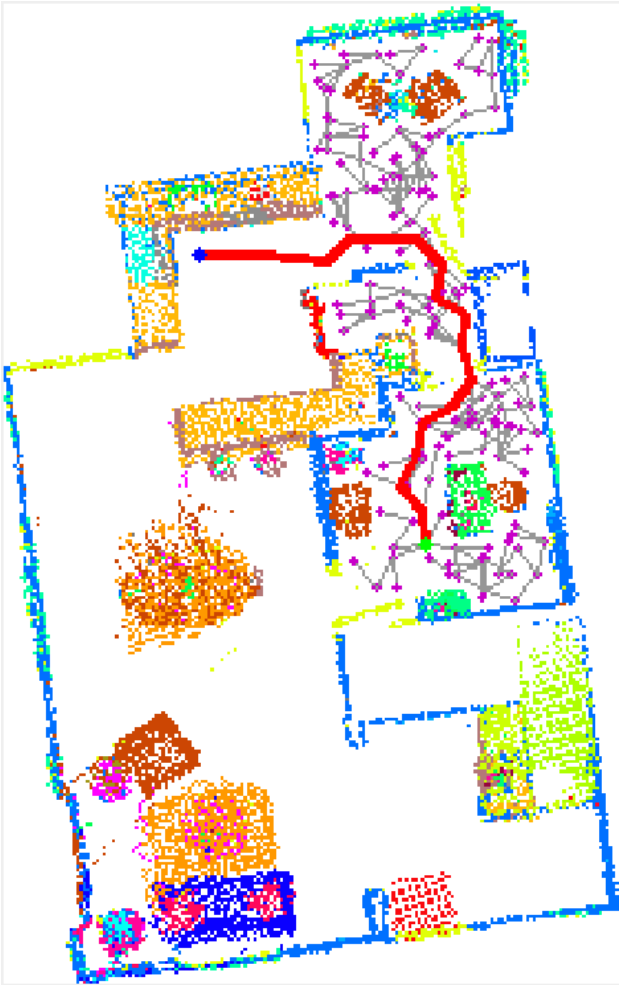

Figure 7: RRT path for the Cooktop.

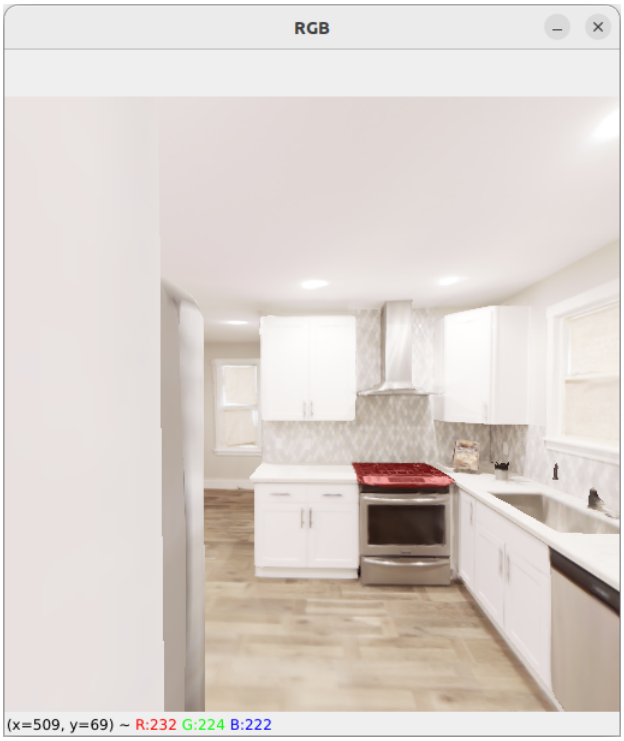

Figure 8: Simulation reaching the cooktop.

Discussion: The RRT tree successfully navigated through the kitchen entrance to reach the cooktop. The occupancy grid's dilation was vital here to maintain a safety buffer from the kitchen counters. The simulation result shows the agent positioned directly in front of the cooktop, with the semantic mask accurately highlighting it.

1.2.3 Case 3: Navigation to the Rack

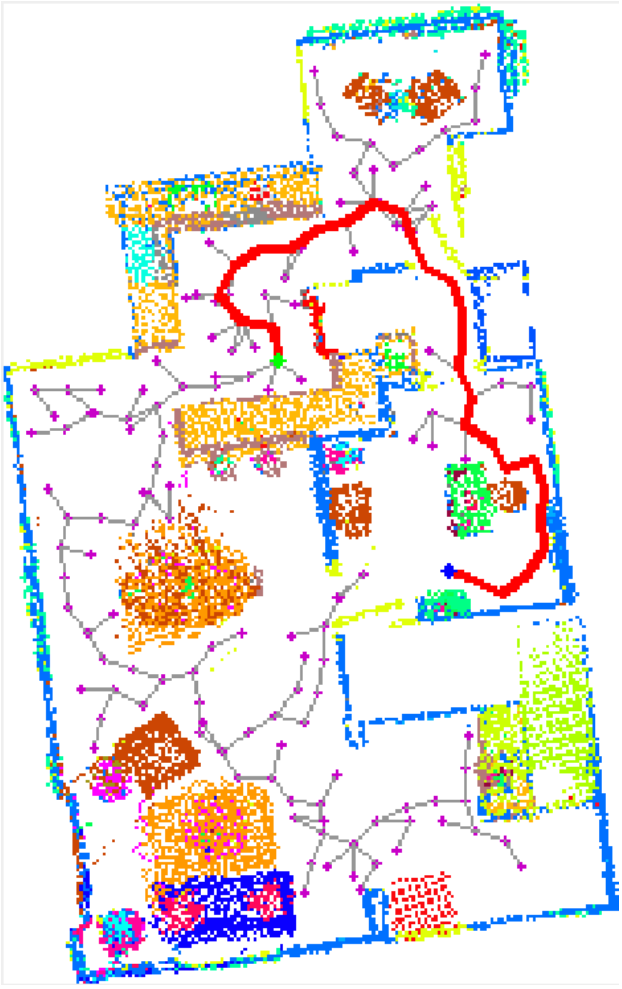

Figure 9: RRT path for the Rack.

Figure 10: Visual confirmation of the rack goal.

Discussion: The rack presents a specific challenge as it is located in a rather small and narrow room. The `find_navigable_goal` function effectively used the distance transform to find a floor point that was close to the rack's centroid and is also reachable (not in a obstacle nor on the other side of the room).

1.2.4 Overall Performance Discussion

- **Efficiency of RRT with Goal Biasing:** The 20% goal bias have saved up the number of iterations needed to find a path compared to a purely random sampling approach. It allowed the tree to grow towards the destination while still providing enough exploration to bypass obstacles.
- **Importance of Control Granularity:** Reducing the `MOVE_AMOUNT` to $0.02m$ was essential for smooth navigation. This smaller step size allowed the agent to follow the RRT waypoints precisely, preventing the overshooting that might occur with the default $0.1m$ step size.

2 Questions

2.1 (a) RRT Parameters: Step Size and Goal Bias

In the RRT algorithm, the sampling behavior and search efficiency are mainly affected by two hyperparameters: **Step Size** and **Goal Bias**.

1. **Step Size:** This defines the maximum distance between a new node and its nearest neighbor in the tree.
    - **Exploration Speed:** A large step size allows the tree to "jump" across open spaces quickly, reaching the goal in fewer iterations. However, if the step size is too large, the algorithm may fail to navigate

through narrow corridors or small doorways because the new node might land inside an obstacle or jump over a valid opening.

- **Precision:** A small step size (like the 10 pixels used in this implementation) provides high precision, allowing the tree to pierce through tight gaps. The trade-off is a significantly higher computational cost, as many more nodes are usually required to cover the same real-world distance.

2. **Goal Bias:** This is the probability of selecting the target goal as the random sample instead of a uniform random point in the environment.

   - **Exploitation (High Bias):** A high bias (e.g., $> 50\%$) makes the algorithm greedy. The tree grows aggressively toward the goal, which is extremely efficient in obstacle-free environments. However, in complex layouts, high bias often leads to the tree getting stuck in local minima, such as U-shaped furniture or dead-end rooms, because it lacks the curiosity to explore the paths that seems to lead away from the goal to find an opening.
   - **Exploration (Low Bias):** A low bias ensures the tree expands uniformly, filling the entire searchable space. While this is slower to converge on a specific target, it is far more robust at finding paths around complex obstacles. My implementation utilized a **20% bias**, which provided a healthy balance: the tree prioritized the target while retaining enough randomness to navigate around the apartment's furniture.

2.2 (b) Real-World Implementation Challenges

Transitioning an indoor navigation pipeline from a well developed simulator like Habitat-Sim to a physical robot involves several critical challenges:

1. **Sensor Noise and Data Sparsity:** Unlike the clean point cloud files used here, real-world depth sensors usually suffer from noises. Transparent surfaces (glass doors), reflective materials (mirrors), and dark objects can create hole-like regions in the point cloud, leading to false obstacles or invisible walls.

2. **Localization Drift:** In simulation, we have access to the Ground Truth position. In reality, robots rely on wheel odometry and IMUs, which accumulate error over time. This drift means the robot's estimated position on the 2D map will eventually diverge from its actual physical location as time goes by, requiring the integration of **SLAM (Simultaneous Localization and Mapping)** to correct its pose.

3. **Dynamic Environments:** The current framework assumes a static world. In a real home, the environment is dynamic: people walk, pets move, and chairs are shifted. A real-world pipeline requires a Local Planner to supplement the RRT, allowing the robot to reactively dodge moving obstacles that were not present in the initial 3D scan.

4. **Computational Constraints:** Real-time processing of high-density 3D point clouds and RRT iterations requires significant CPU/GPU computational power. On a mobile robot with limited battery and edge-computing hardware, the perception pipeline must be highly optimized to maintain a responsive control loop.